TRADINGLAB 2.0 ? DOCUMENTO DE ARQUITECTURA

TradingLab 2.0 — Guía de decisiones y conceptos de arquitectura

Fundamentos, alternativas y consecuencias de las decisiones adoptadas

Versión: 1.2
Estado: Documento de apoyo para validación
Fecha: 7 de octubre de 2026
Documento relacionado: `tradinglab-architecture-v2.md`

1. Propósito

Este documento explica, en lenguaje de decisión, los elementos definidos fuera de la estructura de directorios. Para cada decisión se detalla qué problema resuelve, por qué fue elegida, qué consecuencias introduce y cuándo convendría revisarla.

No reemplaza futuros ADR individuales. Funciona como guía de discusión para validar con los socios el modelo completo antes de implementarlo.

2. Objetivo del sistema

TradingLab 2.0 no será solamente un archivo de robots o reportes. Será un sistema de evidencia experimental capaz de responder:

- qué se intentó demostrar;
- con qué estrategia y versión;
- qué datos, herramientas y supuestos se utilizaron;
- qué resultados se obtuvieron;
- dónde están los archivos originales;
- qué decisiones se tomaron después;
- si el resultado puede verificarse o repetirse.

La arquitectura prioriza trazabilidad y reproducibilidad sobre velocidad inicial de acumulación de resultados.

3. Un único repositorio Git

Decisión

Mantener código, configuraciones, migraciones y documentación de TradingLab en un monorepo.

Motivo

Los componentes evolucionan juntos. Un cambio en el parser puede requerir una migración, una actualización documental y nuevos fixtures. Un único commit permite revisar esa modificación como una unidad coherente.

Alternativas descartadas inicialmente

- Un repositorio por proyecto de trading.
- Un repositorio para software y carpetas no versionadas para documentación.
- Ramas permanentes para representar experimentos.

Estas alternativas fragmentan la trazabilidad o utilizan Git para un problema que corresponde al catálogo experimental.

Consecuencias

- Reglas y CI comunes.
- Referencias simples entre documentos y código.
- Necesidad de excluir correctamente artefactos pesados.

- Necesidad de proteger fronteras internas para evitar acoplamiento.

Cuándo reconsiderar

Cuando existan equipos, ciclos de release, permisos o requisitos regulatorios realmente independientes.

4. Un catálogo lógico central

Decisión

Utilizar una base lógica compartida para proyectos, estrategias, experimentos, runs, métricas y artefactos.

Motivo

Permite comparar proyectos, reutilizar definiciones y contestar preguntas globales sin integrar manualmente bases aisladas.

Qué significa "lógica"

La decisión se refiere al modelo común. El piloto podría comenzar localmente y migrar después a PostgreSQL. Las aplicaciones no deben depender de detalles físicos que impidan esa evolución.

Consecuencias

- Identificadores y vocabulario comunes.
- Migraciones coordinadas.
- Mayor facilidad para análisis transversal.
- La base se vuelve un activo crítico que necesita backups y control de acceso.

Cuándo reconsiderar

Si aparecen restricciones legales, geográficas o de seguridad que obliguen a aislar datos, o si una escala extrema requiere partición física.

5. Separación Git, catálogo y storage

Decisión

Asignar una responsabilidad distinta a cada sistema:

- Git conserva definiciones versionables.
- El catálogo conserva relaciones y estado operativo.
- Storage conserva archivos y datos pesados.

Motivo

Ninguna de estas tecnologías resuelve bien los tres problemas. Git no es una base de resultados ni un almacén de datasets; PostgreSQL no es el lugar ideal para miles de blobs; un filesystem no reemplaza consultas relacionales.

Consecuencia principal

Las referencias entre sistemas deben verificarse. Un artefacto en storage tendrá registro, URI y hash en el catálogo; una versión de estrategia tendrá commit y hash de configuración.

6. Matriz de fuentes de verdad

Decisión

Cada tipo de información tendrá una fuente autoritativa declarada.

Motivo

Sin esta regla aparecen preguntas imposibles de resolver: ¿la métrica válida es la del PDF, la hoja de cálculo o la base? ¿la configuración correcta es el preset exportado o el YAML del repositorio?

Ejemplos

- Código: Git.
- Estado del run: catálogo.
- Archivo original: storage.
- Decisión arquitectónica: ADR.
- Contexto cronológico: bitácora.
- Secreto: gestor de secretos o entorno seguro.

Regla

Una copia derivada debe identificar su fuente y no puede modificarla implícitamente.

7. Proyecto, estrategia, versión, experimento y run

Project

Agrupa una iniciativa de investigación con objetivo y alcance. Puede contener múltiples estrategias.

Strategy

Representa la identidad estable de una idea operativa, aunque cambien sus parámetros o implementación.

StrategyVersion

Congela una implementación y configuración. Dos ejecuciones pueden compararse correctamente porque apuntan a versiones explícitas.

Experiment

Describe la pregunta o hipótesis y el protocolo de evaluación. Ejemplo: comprobar si una estrategia conserva robustez al aumentar costes y desplazar el periodo OOS.

Run

Es una ejecución concreta del experimento. Un experimento puede tener muchos runs por semillas, datasets, motores o escenarios.

Motivo de la separación

Si experimento y run fueran lo mismo, se duplicaría la hipótesis cada vez que cambia una semilla o escenario. Si estrategia y versión fueran lo mismo, se perdería continuidad histórica.

7.1 BotBuild, Environment, Deployment e incubación

Decisión

Representar formalmente el proceso que transforma una `StrategyVersion` de SQX en un bot ejecutable e incubado en MT5.

BotBuild

Identifica una construcción concreta y conserva exportación original, receta de preparación, código preparado, compilador, `.ex5`, log, preset, hashes y resultado de los gates. Dos binarios diferentes son builds diferentes aunque provengan de la misma estrategia.

Environment

Describe el contexto operativo: build de MT5, broker, servidor y cuenta mediante alias, zona horaria, símbolos y máquina/VPS. No contiene credenciales.

Deployment

Declara qué `BotBuild` aprobado fue instalado en qué `Environment`, con qué preset, Magic Number, símbolo y timeframe.

Incubación

Se modela como un experimento de tipo `incubation`; cada periodo de observación es un run vinculado al deployment. Cambiar el ejecutable o preset exige cerrar el run e iniciar otro.

Motivo

Una estrategia no es lo mismo que su binario. Entre SQX y MT5 existen preparación, dependencias, compilación y configuración capaces de alterar el comportamiento. Registrar únicamente `StrategyVersion` y `Run` dejaría ese tramo sin trazabilidad.

Consecuencias

- Se puede identificar exactamente el `.ex5` ejecutado.
- La compilación se vuelve reproducible y auditable.
- Los fallos y builds rechazados se conservan.
- La incubación solo admite builds que superaron gates explícitos.
- Se añaden entidades y procedimientos, pero se elimina ambigüedad operativa.

Fuente detallada

El contrato completo está en `sqx-mt5-incubation-pipeline.md`.

7.2 VPS MT5 aislada y runner local

Decisión

Mantener la VPS MT5 como zona externa sin acceso remoto desde TradingLab y ejecutar allí un componente local, `TradingLab MT5 Runner`, para automatizar tareas aprobadas.

Motivo

La automatización no requiere compartir credenciales ni abrir administración remota. El runner puede compilar, ejecutar, observar y empaquetar resultados dentro de la zona MT5, mientras TradingLab conserva control sobre contratos, versiones y reconciliación.

Intercambio

- `RunnerRelease`: versión instalable del runner.
- `JobPackage`: solicitud declarativa; no contiene comandos arbitrarios.
- `Acknowledgement`: confirmación de recepción, aprobación y estado.
- `EvidenceBundle`: manifiestos, hashes, logs y reportes devueltos.

Google Drive transporta inicialmente esos paquetes. Las copias autoritativas viven en storage y catálogo.

Por qué no una rama de servidor

Una rama permanente no limita el acceso al repositorio, puede divergir de `main` y no identifica con claridad qué versión está instalada. Se mantiene el código en `main` y se producen releases versionados y reversibles.

Consecuencias

- No existe control remoto directo de MT5.
- La aprobación ocurre localmente en la VPS externa.
- Los estados llegan de manera asíncrona.
- Se necesita reconciliar lo solicitado con la evidencia recibida.
- La indisponibilidad temporal de Drive se maneja mediante colas locales.
- En el futuro pueden adoptarse releases inmutables y attestations de GitHub sin cambiar la frontera.

Fuente detallada

El workflow completo está en `tradinglab-mt5-runner-workflow.md`.

8. Datasets y versiones de datos

Decisión

Tratar los datos de mercado como entradas versionadas, no como archivos intercambiables por nombre.

Una DatasetVersion debe identificar

- origen y licencia;
- instrumento y granularidad;
- periodo;
- zona horaria;
- transformaciones;
- calidad y anomalías conocidas;
- hash o manifiesto de hashes;
- versión precedente, si existe.

Motivo

Dos archivos llamados `XAUUSD_M5.csv` pueden tener ticks, zona horaria, correcciones o spreads distintos y producir resultados diferentes.

Consecuencia

Los datasets no necesitan vivir en Git, pero su identidad y procedencia deben estar registradas.

9. Run manifest

Decisión

Cada run tendrá un manifiesto inmutable con todas las entradas necesarias para interpretarlo y, cuando sea posible, repetirlo.

Por qué el commit no alcanza

El commit no informa si había cambios locales, qué build de MT5/SQX se usó, qué dataset exacto se cargó ni qué supuestos de costes se aplicaron.

Grupos de información

- Identidad: run, experimento y estrategia.
- Código: commit y estado limpio/sucio.
- Motor: nombre, versión y build.
- Datos: versión, hash y zona horaria.
- Configuración: hash y parámetros efectivos.
- Mercado: símbolo, spread, comisión y slippage.
- Ejecución: tiempos, semilla y entorno.
- Outputs: artefactos, roles y hashes.

Consecuencia

Un run incompleto podrá conservarse como fallido o inválido, pero no se presentará como reproducible.

10. Artefactos inmutables y direccionados por hash

Decisión

Registrar cada archivo relevante mediante URI, SHA-256, tamaño, media type, rol, backend y productor.

Motivo

Una ruta local puede cambiar y un archivo puede reemplazarse silenciosamente. El hash identifica el contenido y permite detectar corrupción o alteraciones.

Inmutabilidad

Corregir un artefacto genera otro registro. No se sobrescribe el original registrado.

Roles posibles

- input;
- dataset;
- configuración efectiva;
- reporte;
- log;
- modelo intermedio;
- exportación;
- evidencia de validación.

Cuándo reconsiderar

El algoritmo de hash o backend puede cambiar, pero el contrato de identidad por contenido debe mantenerse.

11. Modelo de métricas

Decisión

Separar la definición de una métrica de sus valores y registrar unidad, tipo, scope y versión de cálculo.

Problema del modelo nombre/valor

`drawdown = 10` es ambiguo: podría ser dinero o porcentaje, balance o equity, IS u OOS, y haber sido calculado con fórmulas distintas.

MetricDefinition

Define:

- nombre canónico;
- descripción;
- tipo y unidad;
- fórmula o versión;
- dirección deseable;
- origen compatible.

Metric

Registra:

- run;
- definición;
- valor tipado;
- scope;
- dimensiones controladas;
- procedencia.

Consecuencia

Las consultas son comparables y auditables. El coste es diseñar un vocabulario y gobernar cambios de fórmula.

12. Estados e invalidación

Decisión

Usar estados explícitos y conservar los fallos o resultados invalidados.

Motivo

Eliminar runs defectuosos oculta información importante y puede sesgar el análisis. La invalidación documentada conserva el historial sin mezclar resultados utilizables.

Reglas

- Las transiciones deben validarse.
- Una invalidación registra motivo, autor y fecha.
- `failed` significa fallo de ejecución; `invalidated` significa resultado existente que no debe usarse.
- Los estados terminales no se reabren silenciosamente.

13. PostgreSQL como objetivo, no como acoplamiento

Decisión

PostgreSQL es el motor preferido para operación concurrente, pero el dominio se implementará detrás de casos de uso y contratos.

Motivo

Permite comenzar con una instalación proporcional al piloto sin diseñar scripts que manipulen tablas directamente por todas partes.

Consecuencia

API, CLI y worker compartirán servicios. Los detalles SQL se concentrarán en adaptadores y migraciones.

Cuándo adoptar PostgreSQL desde el inicio

- múltiples usuarios o agentes concurrentes;
- servicio API persistente;
- necesidad de permisos y auditoría;
- automatizaciones que operan simultáneamente.

14. API y CLI antes de MCP

Decisión

Definir operaciones de dominio tipadas y exponerlas primero mediante CLI/API. MCP será un adaptador posterior.

Motivo

Una herramienta específica para agentes no debe convertirse en la única forma de operar el laboratorio. Los mismos casos de uso deben servir a personas, CI y otros sistemas.

Operaciones ejemplo

- crear proyecto;
- registrar versión de estrategia;
- preparar o cerrar run;
- adjuntar artefacto;
- importar resultados;
- consultar estrategias candidatas;
- verificar integridad.

Seguridad

MCP no ofrecerá SQL arbitrario ni acceso irrestricto al filesystem. Las operaciones sensibles utilizarán permisos y aprobaciones.

15. Importación idempotente

Decisión

Importar el mismo archivo más de una vez no debe duplicar el run, sus métricas o artefactos.

Mecanismos

- hash del original;
- claves de idempotencia;
- restricciones únicas;
- transacciones;
- versión del parser;
- estado de importación.

Motivo

Los procesos automáticos se reintentan. Sin idempotencia, un fallo de red o una duda del operador corrompe los conteos.

16. Original, modelo intermedio y métricas normalizadas

Decisión

La ingestión conservará tres niveles cuando sea necesario:

1. Archivo original inmutable.
2. Representación intermedia versionada del parser.
3. Entidades y métricas normalizadas en el catálogo.

Motivo

Si cambia el parser, se pueden recalcular los niveles derivados sin perder la evidencia original. El modelo intermedio también facilita comparar versiones de formatos MT5/SQX.

17. Documentación, ADR y bitácora

Decisión

Separar tres tipos de narrativa:

- Arquitectura: describe el sistema acordado.
- ADR: explica una decisión importante y sus alternativas.
- Bitácora: registra qué ocurrió durante una sesión.

Motivo

Una bitácora cronológica es útil para reconstruir contexto, pero difícil de consultar como política vigente. Un ADR conserva el razonamiento sin reescribir el pasado.

Regla

La bitácora enlaza la decisión; el ADR la fundamenta; la arquitectura refleja el resultado vigente.

18. Seguridad y secretos

Decisión

Mantener secretos fuera de Git y aplicar mínimo privilegio a DB, storage, API y automatizaciones.

Controles principales

- `.env.example` sin valores reales;
- credenciales mediante entorno o gestor de secretos;
- identidades separadas para lectura, ingestión y migración;
- auditoría de operaciones críticas;

- aprobación humana para acciones irreversibles;
- sanitización de archivos externos antes de procesarlos con agentes;
- revisión de licencias de datos de mercado.

Motivo

Un repositorio privado no es un gestor de secretos. Los reportes y datasets también pueden contener datos sensibles o sujetos a licencia.

19. Backups e integridad

Decisión

Respaldar catálogo y storage como un conjunto coordinado y probar restauraciones.

Motivo

Recuperar solo la base dejaría referencias a archivos inexistentes; recuperar solo archivos perdería sus relaciones y estado.

Controles

- backups con versión de esquema;
- checksums;
- cifrado cuando corresponda;
- pruebas de restauración;
- verificación periódica de URIs y hashes;
- definición futura de RPO y RTO.

20. Observabilidad

Decisión

Emitir logs estructurados y correlacionar operaciones por identificadores de dominio.

Campos de correlación

- `correlation_id`;
- `project_id`;
- `experiment_id`;
- `run_id`;
- versión de parser o servicio.

Motivo

Cuando una importación falle debe ser posible seguir su recorrido sin buscar manualmente en archivos inconexos.

21. Calidad y pruebas

Decisión

Validar reglas del dominio, parsers, migraciones y al menos un flujo real de extremo a extremo.

Niveles

- tests unitarios de reglas y transformaciones;
- fixtures representativas y anonimizadas de MT5/SQX;
- tests de regresión de parsers;
- pruebas de migración;
- test de integración con DB/storage;
- test end-to-end del piloto.

Motivo

Los parsers y formatos externos son especialmente propensos a cambios silenciosos. Un resultado financiero incorrecto puede parecer plausible y pasar inadvertido.

22. Codex, AGENTS y skills

Decisión

Usar agentes dentro de límites explícitos y con procedimientos versionados.

Reparto de responsabilidades

- `AGENTS.md`: reglas que siempre aplican.
- Skills: workflows activados por contexto.
- Scripts: mecánica determinista.
- API/CLI: capacidades controladas del sistema.
- Personas: aprobación de decisiones y acciones de riesgo.

Motivo

Esto permite aprovechar automatización sin depender de memoria conversacional ni otorgar acceso indiscriminado.

Restricción

La arquitectura debe seguir siendo operable sin Codex. Los documentos, contratos y herramientas pertenecen a TradingLab.

23. Implementación mediante flujo vertical

Decisión

Probar primero un caso completo con un proyecto y un reporte real, en lugar de implementar todas las tablas y servicios teóricos.

Flujo piloto

```
Proyecto → Estrategia → Versión → Experimento → Run
        → Reporte MT5/SQX → Artefacto → Métricas → Consulta
```

Motivo

Un flujo vertical revela requisitos reales de formatos, identidad, errores y operación. Reduce el riesgo de construir una plataforma extensa basada en supuestos.

Criterio de éxito

Poder navegar desde una métrica hasta su reporte original, verificar el hash y conocer código, configuración, datos, motor y contexto de decisión.

24. Qué se pospone deliberadamente

- interfaz gráfica completa;
- sistema distribuido de workers;
- almacenamiento de objetos administrado;
- MCP;
- analítica avanzada y dashboards;
- múltiples bases físicas;
- catálogo universal de todas las métricas;
- automatización completa de MT5/SQX.

Posponer no significa descartar. Significa evitar que componentes secundarios bloqueen la validación del núcleo.

25. Decisiones que requieren acuerdo de socios

Decisión	Impacto
Propietario y visibilidad de GitHub	Control, colaboración y confidencialidad
Motor del catálogo durante el piloto	Complejidad operativa y concurrencia
Storage inicial y backups	Seguridad y durabilidad de evidencia
Primer proyecto piloto	Calidad de la validación arquitectónica
Tecnología de API/CLI	Ecosistema de desarrollo y mantenimiento
Política de ramas y revisiones	Velocidad frente a control
Retención de artefactos y datasets	Coste, auditoría y reproducibilidad
Permisos de agentes	Riesgo operativo y grado de automatización
Licencias de datos	Qué puede almacenarse y compartirse

26. Glosario breve

- **ADR:** registro versionado de una decisión arquitectónica.
- **Artefacto:** archivo producido o consumido y registrado por el laboratorio.
- **Catálogo:** base estructurada de entidades, relaciones, estados y métricas.
- **Hash:** huella del contenido utilizada para verificar identidad e integridad.
- **Idempotencia:** propiedad por la cual repetir una operación no duplica sus efectos.
- **Manifiesto:** descripción estructurada de entradas, entorno y outputs de un run.
- **MCP:** protocolo que puede exponer herramientas controladas a agentes.
- **RPO:** pérdida máxima de datos aceptable medida en tiempo.
- **RTO:** tiempo objetivo para restaurar el servicio.
- **Scope:** contexto en el que se calculó una métrica, como IS u OOS.
- **Storage:** sistema que conserva datasets y artefactos binarios.
- **Trazabilidad:** capacidad de recorrer relaciones entre resultados, entradas y decisiones.

27. Resumen para presentar a socios

La propuesta introduce más disciplina que una estructura tradicional de carpetas porque TradingLab gestiona evidencia experimental. Sus decisiones más importantes son:

1. Mantener una historia coherente en un único repositorio.
2. Separar definiciones, metadatos y archivos pesados.
3. Identificar de forma inmutable datos, configuraciones, ejecuciones y artefactos.
4. Hacer comparables las métricas mediante definiciones y contexto.
5. Conservar tanto éxitos como fallos e invalidaciones.
6. Automatizar mediante operaciones controladas y auditables.
7. Validar la arquitectura con un flujo real antes de escalarla.

El coste inicial es acordar vocabulario y procedimientos. El beneficio es evitar que, con el crecimiento, los resultados se conviertan en archivos difíciles de verificar, comparar o reproducir.